EXPERIMENT NO: 09

Experiment Title: Implement localStorage and sessionStorage to save user preferences like theme settings.

Software/Tools Required:

- Any text editor / VS Code
- Web Browser (Chrome/Edge) with Developer Console

Theory :

Web Storage API: The Web Storage API allows web applications to store data in the user's browser. JavaScript provides two types of storage: localStorage and sessionStorage. These storage mechanisms help save user preferences, such as theme settings, without requiring a server or database.

Types of Web Storage:

1. **localStorage:** Stores data persistently in the browser. The saved data remains available even after the browser is closed and reopened, until it is explicitly removed or cleared.
2. **sessionStorage:** Stores data for the current browser tab session. The data survives page refreshes but is normally cleared when the tab session ends.

Key Storage Methods:

- setItem(key, value): Stores a key-value pair.
- getItem(key): Retrieves a stored value.
- removeItem(key): Removes a specific stored item.
- clear(): Removes all items from the selected storage area.

Theme Preference Management: JavaScript can access HTML elements using DOM methods such as document.getElementById(). By using event listeners, users can select a theme and save their preferences. The classList.toggle() method applies or removes the dark theme CSS class dynamically.

Implementation: In this experiment, an HTML page is designed with light and dark theme options, storage selection, a save button, and a reset button. JavaScript stores and retrieves the selected theme using either localStorage or sessionStorage. The saved preference is applied when the page loads, demonstrating persistent and session-based user preference management.

Experiment Program Code

Code 1: prefernce.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>User Preferences Storage</title>
```

```
<style>
  * {
    box-sizing: border-box;
  }

  body {
    font-family: Arial, sans-serif;
    background: #f4f6f9;
    color: #222;
    margin: 0;
    padding: 20px;
    transition: 0.3s;
  }

  .container {
    max-width: 500px;
    margin: 50px auto;
    padding: 30px;
    background: white;
    border-radius: 12px;
    box-shadow: 0 4px 15px #0002;
  }

  h1 {
    text-align: center;
    font-size: 26px;
  }

  p {
    line-height: 1.6;
  }

  label {
    display: block;
    margin-top: 20px;
    font-weight: bold;
  }

  select, button {
    width: 100%;
    padding: 12px;
    margin-top: 10px;
    border-radius: 6px;
    font-size: 16px;
  }

  select {
    border: 1px solid #bbb;
    background: white;
    color: #222;
  }

  button {
    background: #2563eb;
    color: white;
    border: none;
    cursor: pointer;
  }

  button:hover {
    opacity: 0.85;
  }

  #status {
    margin-top: 20px;
    padding: 12px;
    background: #e0f2fe;
    color: #075985;
    border-radius: 6px;
    overflow-wrap: anywhere;
  }

  /* Dark theme */
  body.dark {
    background: #111827;
    color: #f9fafb;
```

```
    }

    body.dark .container {
      background: #1f2937;
    }

    body.dark select {
      background: #374151;
      color: white;
      border-color: #6b7280;
    }

    body.dark #status {
      background: #374151;
      color: #bfdbfe;
    }
  </style>
</head>

<body>
  <div class="container">
    <h1>User Preferences</h1>

    <p>
      Choose your preferred theme and how you want
      your settings to be stored.
    </p>

    <label for="theme">Select Theme:</label>
    <select id="theme">
      <option value="light">Light Theme</option>
      <option value="dark">Dark Theme</option>
    </select>

    <label for="storage">Save Preference Using:</label>
    <select id="storage">
      <option value="local">localStorage (Permanent)</option>
      <option value="session">sessionStorage (Current Tab)</option>
    </select>

    <button id="saveBtn">Save Preferences</button>
    <button id="resetBtn">Reset Preferences</button>

    <div id="status" role="status" aria-live="polite">
      Your preferences will appear here.
    </div>
  </div>

  <script>
    const themeSelect = document.getElementById("theme");
    const storageSelect = document.getElementById("storage");
    const status = document.getElementById("status");

    // Apply a theme to the page
    function applyTheme(theme) {
      document.body.classList.toggle("dark", theme === "dark");
      themeSelect.value = theme;
    }

    // Save the user's selected preferences
    function savePreferences() {
      const theme = themeSelect.value;
      const storageType = storageSelect.value;

      // Remove the old saved theme from both storage types
      localStorage.removeItem("theme");
      sessionStorage.removeItem("theme");

      // Save the theme using the selected storage type
      if (storageType === "local") {
        localStorage.setItem("theme", theme);
      } else {
        sessionStorage.setItem("theme", theme);
      }

      // Save the selected storage method as well
      localStorage.setItem("storageType", storageType);
```

```
      applyTheme(theme);

      status.textContent =
        "Theme saved successfully using " +
        (storageType === "local"
          ? "localStorage."
          : "sessionStorage.");
    }

    // Load previously saved preferences
    function loadPreferences() {
      const savedStorage = localStorage.getItem("storageType") || "local";

      storageSelect.value = savedStorage;

      let savedTheme;

      if (savedStorage === "session") {
        savedTheme = sessionStorage.getItem("theme");
      } else {
        savedTheme = localStorage.getItem("theme");
      }

      // Use light theme if no preference exists
      savedTheme = savedTheme || "light";

      applyTheme(savedTheme);

      status.textContent =
        "Current theme: " + savedTheme +
        ". Storage: " + savedStorage + ".";
    }

    // Reset saved preferences
    function resetPreferences() {
      localStorage.removeItem("theme");
      localStorage.removeItem("storageType");
      sessionStorage.removeItem("theme");

      themeSelect.value = "light";
      storageSelect.value = "local";

      applyTheme("light");

      status.textContent = "Preferences reset to default.";
    }

    document.getElementById("saveBtn")
      .addEventListener("click", savePreferences);

    document.getElementById("resetBtn")
      .addEventListener("click", resetPreferences);

    // Restore preferences when the page opens
    loadPreferences();
  </script>
</body>
</html>
```

Code 2: theme.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>

</body>
```

```
</html>
```

5. Case Study Title

Design a gym admission form with live input checks.

6. Case Study Program Code

Casestudy_09_Kanak_Tembhurne_1290.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Experiment 9 - Theme Settings</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
            transition: background-color 0.3s, color 0.3s;
        }

        /* Light Theme Styles */
        body.light {
            background-color: #f4f4f9;
            color: #333333;
        }

        body.light .card {
            background-color: #e6e6e6;
            border: 1px solid #b0b0b0;
        }

        /* Dark Theme Styles */
        body.dark {
            background-color: #1e1e1e;
            color: #ffffff;
        }

        body.dark .card {
            background-color: #4a3b32;
            border: 1px solid #6b5245;
        }

        .card {
            padding: 30px 40px;
            border-radius: 4px;
            text-align: center;
            width: 320px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
        }

        h2 {
            margin-top: 0;
            font-size: 22px;
        }

        label {
            display: block;
            margin-bottom: 8px;
```

```
            font-size: 14px;
        }

        select {
            padding: 4px 8px;
            margin-bottom: 20px;
            font-size: 14px;
            border-radius: 4px;
        }

        .button-group {
            display: flex;
            justify-content: space-between;
            margin-bottom: 12px;
            gap: 8px;
        }

        button {
            padding: 6px 10px;
            font-size: 12px;
            cursor: pointer;
            border: 1px solid #ccc;
            border-radius: 4px;
            background-color: #ffffff;
            color: #333;
        }

        button:hover {
            background-color: #f0f0f0;
        }

        .clear-btn {
            width: 100%;
            margin-bottom: 15px;
        }

        .status-msg {
            font-size: 13px;
            min-height: 18px;
            margin: 0;
        }
    </style>
</head>
<body class="light">

    <div class="card">
        <h2>Theme Settings</h2>

        <label for="themeSelect">Select your preferred theme:</label>
        <select id="themeSelect" onchange="previewTheme()">
            <option value="light">Light</option>
            <option value="dark">Dark</option>
        </select>

        <div class="button-group">
            <button onclick="saveLocalStorage()">Save using localStorage</button>
            <button onclick="saveSessionStorage()">Save using sessionStorage</button>
        </div>

        <button class="clear-btn" onclick="clearPreferences()">Clear Preferences</button>

        <p id="statusMsg" class="status-msg"></p>
    </div>

    <script>
        const themeSelect = document.getElementById('themeSelect');
        const statusMsg = document.getElementById('statusMsg');

        // Apply temporary theme dynamically upon changing select dropdown
        function previewTheme() {
            document.body.className = themeSelect.value;
        }

        // Save theme in localStorage (persists when tab/browser closes)
        function saveLocalStorage() {
            const selectedTheme = themeSelect.value;
            // Clear sessionStorage to avoid conflicts
```

```
            sessionStorage.removeItem('theme');

            localStorage.setItem('theme', selectedTheme);
            statusMsg.textContent = 'Theme saved using localStorage.';
        }

        // Save theme in sessionStorage (resets to default light when reloaded or reopened)
        function saveSessionStorage() {
            const selectedTheme = themeSelect.value;
            // Clear localStorage to avoid persistence override
            localStorage.removeItem('theme');

            sessionStorage.setItem('theme', selectedTheme);
            statusMsg.textContent = 'Theme saved using sessionStorage.';
        }

        // Clear all saved theme preferences and reset to Light
        function clearPreferences() {
            localStorage.removeItem('theme');
            sessionStorage.removeItem('theme');

            document.body.className = 'light';
            themeSelect.value = 'light';
            statusMsg.textContent = 'Preferences cleared.';
        }

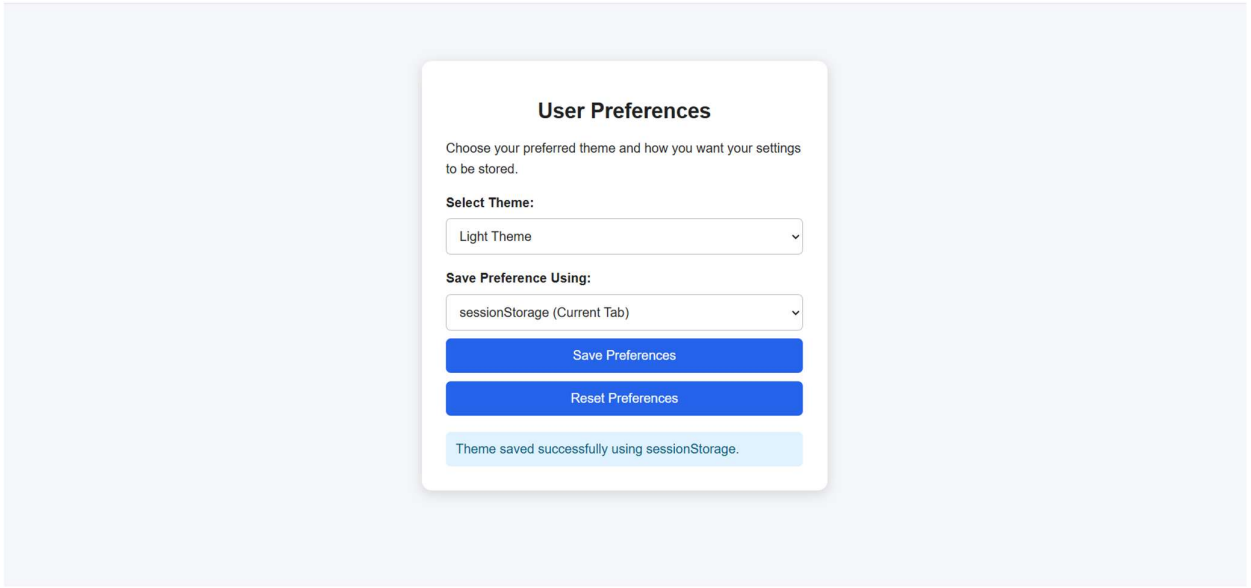
        // Load saved theme on initial page load
        function loadTheme() {
            const localTheme = localStorage.getItem('theme');
            const sessionTheme = sessionStorage.getItem('theme');

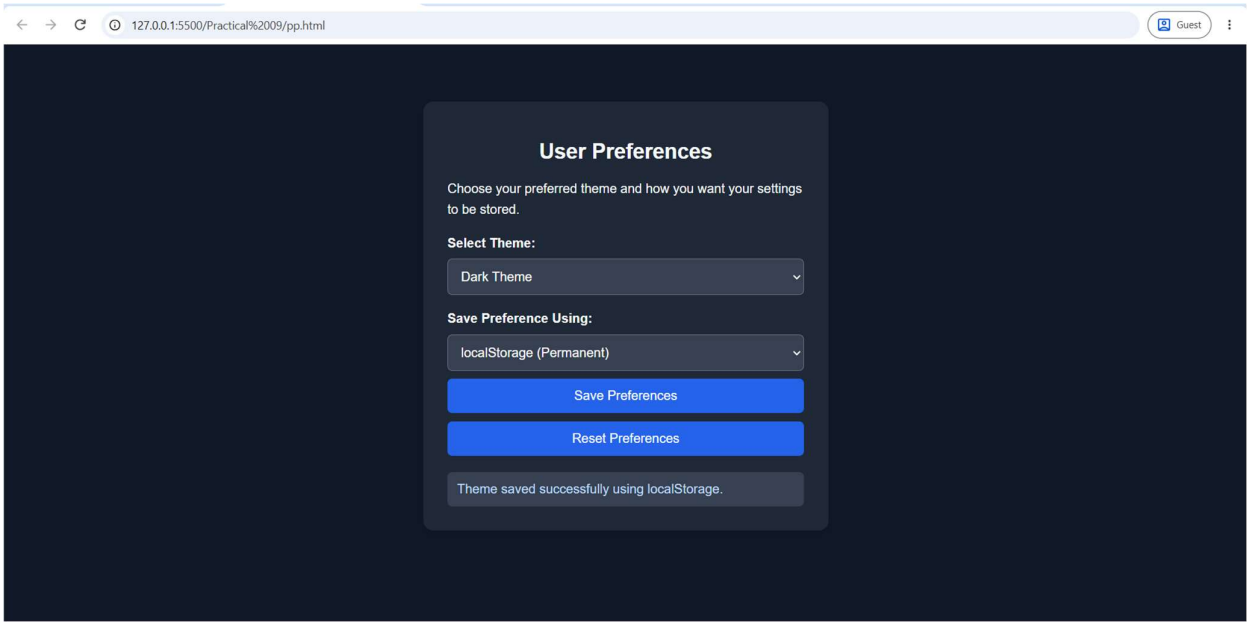
            if (localTheme) {
                // Persistent state restored
                document.body.className = localTheme;
                themeSelect.value = localTheme;
                statusMsg.textContent = 'Theme loaded from localStorage.';
            } else if (sessionTheme) {
                // Session state applied
                document.body.className = sessionTheme;
                themeSelect.value = sessionTheme;
                statusMsg.textContent = 'Theme saved using sessionStorage.';
            } else {
                // Default theme when session ends, reloaded, or cleared
                document.body.className = 'light';
                themeSelect.value = 'light';
                statusMsg.textContent = '';
            }
        }

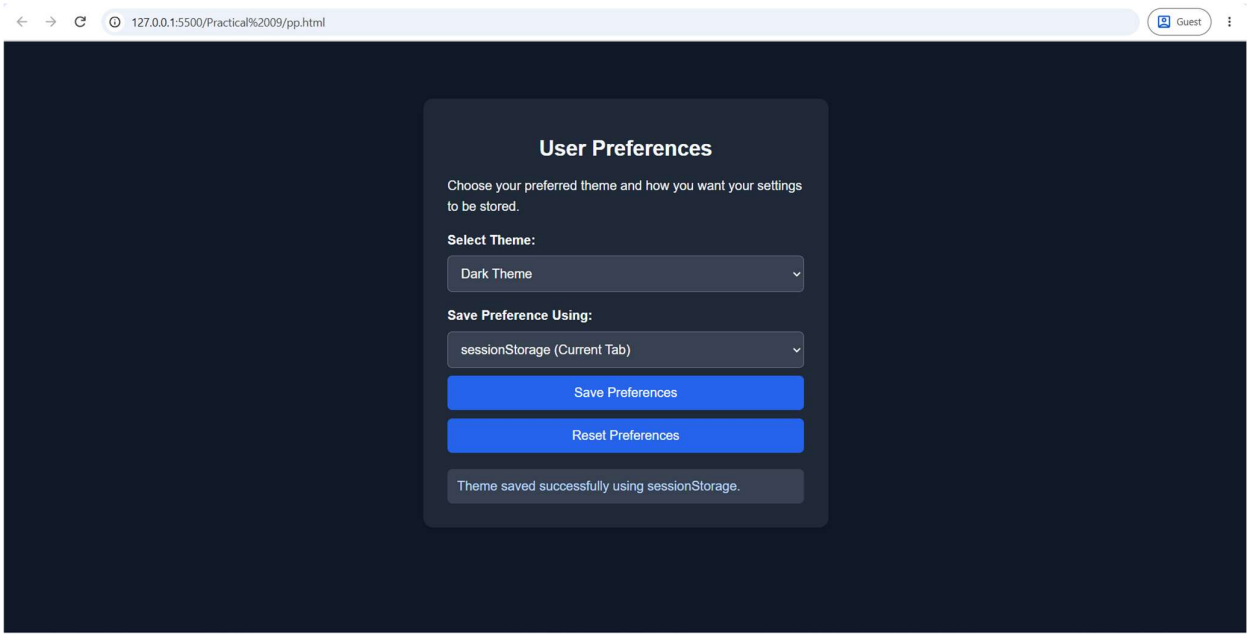
        // Run theme loader on startup
        loadTheme();
    </script>
</body>
</html>
```

7. Output of the case study :

Name : Kanak Tembhurne PRN : 24070521290 File path :
http://127.0.0.1:5500/Practical%2002/Casestudy_09_Kanak_Tembhurne_1290.html


User Preferences
Choose your preferred theme and how you want your settings to be stored.
Select Theme:
Light Theme
Save Preference Using:
sessionStorage (Current Tab)
Save Preferences
Reset Preferences
Theme saved successfully using sessionStorage.


127.0.0.1:5500/Practical%2009/pp.html
Guest
User Preferences
Choose your preferred theme and how you want your settings to be stored.
Select Theme:
Dark Theme
Save Preference Using:
localStorage (Permanent)
Save Preferences
Reset Preferences
Theme saved successfully using localStorage.



8. Conclusion

Thus, the implementation of localStorage and sessionStorage was successfully demonstrated using JavaScript to save, retrieve, and reset user preferences, including light and dark theme settings.